Name : S.Bala Yokesh
Class : IT Batch 2
Roll no : 2025115080

DS Homework Set – 5

1. Use linked list for polynomial representation and perform addition of selected operations on the polynomials.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int coeff;
  int exp;
  struct Node *next;
};

struct Node* createNode(int coeff, int exp) {
  struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));

  newNode->coeff = coeff;
  newNode->exp = exp;
  newNode->next = NULL;

  return newNode;
}

void insert(struct Node **head, int coeff, int exp) {
  struct Node *newNode = createNode(coeff, exp);

  if (*head == NULL) {
    *head = newNode;
    return;
  }

  struct Node *temp = *head;

  while (temp->next != NULL)
    temp = temp->next;

  temp->next = newNode;
}

struct Node* addPolynomial(struct Node *p1, struct Node *p2) {
  struct Node *result = NULL;

  while (p1 != NULL && p2 != NULL) {
    if (p1->exp == p2->exp) {
      insert(&result, p1->coeff + p2->coeff, p1->exp);
      p1 = p1->next;
      p2 = p2->next;
```

```
    }
    else if (p1->exp > p2->exp) {
      insert(&result, p1->coeff, p1->exp);
      p1 = p1->next;
    }
    else {
      insert(&result, p2->coeff, p2->exp);
      p2 = p2->next;
    }
  }

  while (p1 != NULL) {
    insert(&result, p1->coeff, p1->exp);
    p1 = p1->next;
  }

  while (p2 != NULL) {
    insert(&result, p2->coeff, p2->exp);
    p2 = p2->next;
  }

  return result;
}

void display(struct Node *head) {
  while (head != NULL) {
    printf("%dx^%d", head->coeff, head->exp);

    if (head->next != NULL)
      printf(" + ");

    head = head->next;
  }

  printf("\n");
}

int main() {
  struct Node *p1 = NULL;
  struct Node *p2 = NULL;
  struct Node *result;

  insert(&p1, 5, 3);
  insert(&p1, 4, 2);
  insert(&p1, 2, 0);

  insert(&p2, 3, 3);
  insert(&p2, 2, 2);
  insert(&p2, 7, 1);

  printf("Polynomial 1: ");
  display(p1);
```

```
    printf("Polynomial 2: ");
    display(p2);

    result = addPolynomial(p1, p2);

    printf("Result: ");
    display(result);

    return 0;
}
```

Output :

```
Polynomial 1: 5x^3 + 4x^2 + 2x^0
Polynomial 2: 3x^3 + 2x^2 + 7x^1
Result: 8x^3 + 6x^2 + 7x^1 + 2x^0
```

2. Find previous node of circular LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = data;

    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != *head)
        temp = temp->next;

    temp->next = newNode;
    newNode->next = *head;
}

void findPrevious(struct Node *head, int key) {
    if (head == NULL)
        return;

    struct Node *temp = head;

    do {
```

```
    if (temp->next->data == key) {
      printf("Previous node of %d is %d\n",
          key, temp->data);
      return;
    }

    temp = temp->next;

  } while (temp != head);

  printf("Element not found\n");
}

int main() {
  struct Node *head = NULL;

  insert(&head, 10);
  insert(&head, 20);
  insert(&head, 30);
  insert(&head, 40);

  findPrevious(head, 30);

  return 0;
}
```

Output :

Previous node of 30 is 20

3. Insert at a given position of a circular LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

void insertAtPosition(struct Node **head, int data, int pos) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;

  if (*head == NULL) {
    newNode->next = newNode;
    *head = newNode;
    return;
  }

  if (pos == 1) {
```

```
    struct Node *temp = *head;

    while (temp->next != *head)
      temp = temp->next;

    newNode->next = *head;
    temp->next = newNode;
    *head = newNode;
    return;
  }

  struct Node *temp = *head;

  for (int i = 1; i < pos - 1; i++) {
    temp = temp->next;

    if (temp == *head) {
      printf("Invalid position\n");
      return;
    }
  }

  newNode->next = temp->next;
  temp->next = newNode;
}

void display(struct Node *head) {
  if (head == NULL)
    return;

  struct Node *temp = head;

  do {
    printf("%d ", temp->data);
    temp = temp->next;
  } while (temp != head);

  printf("\n");
}

int main() {
  struct Node *head = NULL;

  insertAtPosition(&head, 10, 1);
  insertAtPosition(&head, 20, 2);
  insertAtPosition(&head, 30, 3);
  insertAtPosition(&head, 15, 2);

  printf("Circular Linked List: ");
  display(head);

  return 0;
}
```

Output :

Circular Linked List: 10 15 20 30

4. Find largest 2nd element in a Queue.

```
#include <stdio.h>

#define MAX 100

int main() {
  int queue[MAX];
  int n;

  printf("Enter number of elements: ");
  scanf("%d", &n);

  printf("Enter elements:\n");

  for (int i = 0; i < n; i++)
    scanf("%d", &queue[i]);

  int largest = queue[0];
  int secondLargest = -999999;

  for (int i = 1; i < n; i++) {

    if (queue[i] > largest) {
      secondLargest = largest;
      largest = queue[i];
    }
    else if (queue[i] > secondLargest &&
         queue[i] != largest) {
      secondLargest = queue[i];
    }
  }

  if (secondLargest == -999999)
    printf("Second largest element does not exist\n");
  else
    printf("Second largest element = %d\n", secondLargest);

  return 0;
}
```

Output :

```
Enter number of elements: 5
Enter elements:
10 50 20 40 30
Second largest element = 40
```

5. Insert 3rd element and check whether 1st and 3rd elements are same in a LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

void insertAtThird(struct Node **head, int data) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;

  if (*head == NULL || (*head)->next == NULL) {
    printf("At least two nodes are required\n");
    free(newNode);
    return;
  }

  newNode->next = (*head)->next->next;
  (*head)->next->next = newNode;
}

void display(struct Node *head) {
  while (head != NULL) {
    printf("%d ", head->data);
    head = head->next;
  }

  printf("\n");
}

int main() {
  struct Node *head = NULL;

  struct Node *a = malloc(sizeof(struct Node));
  struct Node *b = malloc(sizeof(struct Node));

  a->data = 10;
  b->data = 20;

  a->next = b;
  b->next = NULL;

  head = a;

  insertAtThird(&head, 10);

  printf("Linked List: ");
  display(head);
```

```
  if (head->data == head->next->next->data)
    printf("1st and 3rd elements are same\n");
  else
    printf("1st and 3rd elements are not same\n");

  return 0;
}
```

Output :

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

void insertAtThird(struct Node **head, int data) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;

  if (*head == NULL || (*head)->next == NULL) {
    printf("At least two nodes are required\n");
    free(newNode);
    return;
  }

  newNode->next = (*head)->next->next;
  (*head)->next->next = newNode;
}

void display(struct Node *head) {
  while (head != NULL) {
    printf("%d ", head->data);
    head = head->next;
  }

  printf("\n");
}

int main() {
  struct Node *head = NULL;

  struct Node *a = malloc(sizeof(struct Node));
  struct Node *b = malloc(sizeof(struct Node));

  a->data = 10;
  b->data = 20;
```

```
  a->next = b;
  b->next = NULL;

  head = a;

  insertAtThird(&head, 10);

  printf("Linked List: ");
  display(head);

  if (head->data == head->next->next->data)
    printf("1st and 3rd elements are same\n");
  else
    printf("1st and 3rd elements are not same\n");

  return 0;
}
```

Output :

```
Linked List: 10 20 10
1st and 3rd elements are same
```

6. Implement two stacks in one array with rotate function to reverse the array and display.

```
#include <stdio.h>

#define MAX 10

int arr[MAX];

int top1 = -1;
int top2 = MAX;

void push1(int value) {
  if (top1 + 1 == top2) {
    printf("Stack Overflow\n");
    return;
  }

  arr[++top1] = value;
}

void push2(int value) {
  if (top1 + 1 == top2) {
    printf("Stack Overflow\n");
    return;
  }

  arr[--top2] = value;
}

int pop1() {
```

```
    if (top1 == -1) {
        printf("Stack 1 Underflow\n");
        return -1;
    }

    return arr[top1--];
}

int pop2() {
    if (top2 == MAX) {
        printf("Stack 2 Underflow\n");
        return -1;
    }

    return arr[top2++];
}

void rotate() {
    int i, j, temp;

    i = 0;
    j = top1;

    while (i < j) {
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;

        i++;
        j--;
    }
}

void display() {
    printf("Array: ");

    for (int i = 0; i <= top1; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

int main() {
    push1(10);
    push1(20);
    push1(30);
    push1(40);

    printf("Before reverse:\n");
    display();

    rotate();
```

```
  printf("After reverse:\n");
  display();

  return 0;
}
```

Output :

Before reverse:
Array: 10 20 30 40

After reverse:
Array: 40 30 20 10

7. Given a linked list containing n head nodes where every node in the LL contains two pointers — one pointer to the next node and another pointer to a sub-linked list. The head nodes are sorted in ascending order based on their data. We need to flatten the LL so that all the nodes appear in a single LL while maintaining sorted order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
  struct Node *child;
};

struct Node* createNode(int data) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->next = NULL;
  newNode->child = NULL;

  return newNode;
}

struct Node* merge(struct Node *a, struct Node *b) {
  if (a == NULL)
    return b;

  if (b == NULL)
    return a;

  struct Node *result;

  if (a->data <= b->data) {
    result = a;
    result->child = merge(a->child, b);
  }
  else {
```

```
    result = b;
    result->child = merge(a, b->child);
  }

  return result;
}

struct Node* flatten(struct Node *head) {
  if (head == NULL || head->next == NULL)
    return head;

  head->next = flatten(head->next);

  head = merge(head, head->next);

  return head;
}

void display(struct Node *head) {
  while (head != NULL) {
    printf("%d ", head->data);
    head = head->child;
  }

  printf("\n");
}

int main() {

  struct Node *head = createNode(5);

  head->next = createNode(10);
  head->next->next = createNode(19);
  head->next->next->next = createNode(28);

  head->child = createNode(7);
  head->child->child = createNode(8);

  head->next->child = createNode(20);

  head->next->next->child = createNode(22);

  head->next->next->next->child = createNode(35);
  head->next->next->next->child->child = createNode(40);

  head = flatten(head);

  printf("Flattened Linked List:\n");
  display(head);

  return 0;
}
```

Output :

Flattened Linked List:
5 7 8 10 19 20 22 28 35 40

8. Given k sorted LL of different sizes, we need to merge them into a single LL maintaining their sorted order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

struct Node* createNode(int data) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->next = NULL;

  return newNode;
}

struct Node* merge(struct Node *a, struct Node *b) {

  if (a == NULL)
    return b;

  if (b == NULL)
    return a;

  struct Node *result;

  if (a->data <= b->data) {
    result = a;
    result->next = merge(a->next, b);
  }
  else {
    result = b;
    result->next = merge(a, b->next);
  }

  return result;
}

void display(struct Node *head) {
  while (head != NULL) {
    printf("%d ", head->data);
    head = head->next;
  }
```

```
  printf("\n");
}

int main() {

  struct Node *list1 = createNode(1);
  list1->next = createNode(4);
  list1->next->next = createNode(7);

  struct Node *list2 = createNode(2);
  list2->next = createNode(5);
  list2->next->next = createNode(8);

  struct Node *list3 = createNode(3);
  list3->next = createNode(6);
  list3->next->next = createNode(9);

  struct Node *result;

  result = merge(list1, list2);
  result = merge(result, list3);

  printf("Merged Sorted Linked List:\n");
  display(result);

  return 0;
}
```

Output :

```
Merged Sorted Linked List:
1 2 3 4 5 6 7 8 9
```

9. Swap kth node from beginning with kth node from end in a LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

struct Node* createNode(int data) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->next = NULL;

  return newNode;
}
```

```
void swapKth(struct Node *head, int k) {

  struct Node *first = head;
  struct Node *second = head;

  int n = 0;

  while (second != NULL) {
    n++;
    second = second->next;
  }

  if (k > n) {
    printf("Invalid k\n");
    return;
  }

  first = head;

  for (int i = 1; i < k; i++)
    first = first->next;

  second = head;

  for (int i = 1; i <= n - k; i++)
    second = second->next;

  int temp = first->data;
  first->data = second->data;
  second->data = temp;
}

void display(struct Node *head) {
  while (head != NULL) {
    printf("%d ", head->data);
    head = head->next;
  }

  printf("\n");
}

int main() {

  struct Node *head = createNode(10);

  head->next = createNode(20);
  head->next->next = createNode(30);
  head->next->next->next = createNode(40);
  head->next->next->next->next = createNode(50);

  int k = 2;
```

```
  printf("Before swapping:\n");
  display(head);

  swapKth(head, k);

  printf("After swapping:\n");
  display(head);

  return 0;
}
```

Output :

```
Before swapping:
10 20 30 40 50

After swapping:
10 40 30 20 50
```

10. Multiply two numbers represented by two LL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

struct Node* createNode(int data) {
  struct Node *newNode =
    (struct Node*)malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->next = NULL;

  return newNode;
}

void insertEnd(struct Node **head, int data) {

  struct Node *newNode = createNode(data);

  if (*head == NULL) {
    *head = newNode;
    return;
  }

  struct Node *temp = *head;

  while (temp->next != NULL)
    temp = temp->next;
```

```
    temp->next = newNode;
}

long long convertToNumber(struct Node *head) {

    long long num = 0;

    while (head != NULL) {
        num = num * 10 + head->data;
        head = head->next;
    }

    return num;
}

int main() {

    struct Node *num1 = NULL;
    struct Node *num2 = NULL;

    insertEnd(&num1, 1);
    insertEnd(&num1, 0);
    insertEnd(&num1, 0);

    insertEnd(&num2, 2);
    insertEnd(&num2, 0);

    long long n1 = convertToNumber(num1);
    long long n2 = convertToNumber(num2);

    printf("First number: %lld\n", n1);
    printf("Second number: %lld\n", n2);

    printf("Product: %lld\n", n1 * n2);

    return 0;
}
```

Output :

```
First number: 100
Second number: 20
Product: 2000
```